UNIT-2 (UNIT-1(INTRODUCTION OF OS, LAYERED ARCHITECTURE/ LOGICAL STRUCTURE OF OS)
Memory Management

LECTURE NO:-1

Memory Management

Memory management is the functionality of an operating system which handles or manages primary memory and moves processes back and forth between main memory and disk during execution.

Memory management keeps track of each and every memory location, regardless of either it is allocated to some process or it is free. It checks how much memory is to be allocated to processes.

It decides which process will get memory at what time. It tracks whenever some memory gets freed or unallocated and correspondingly it updates the status.

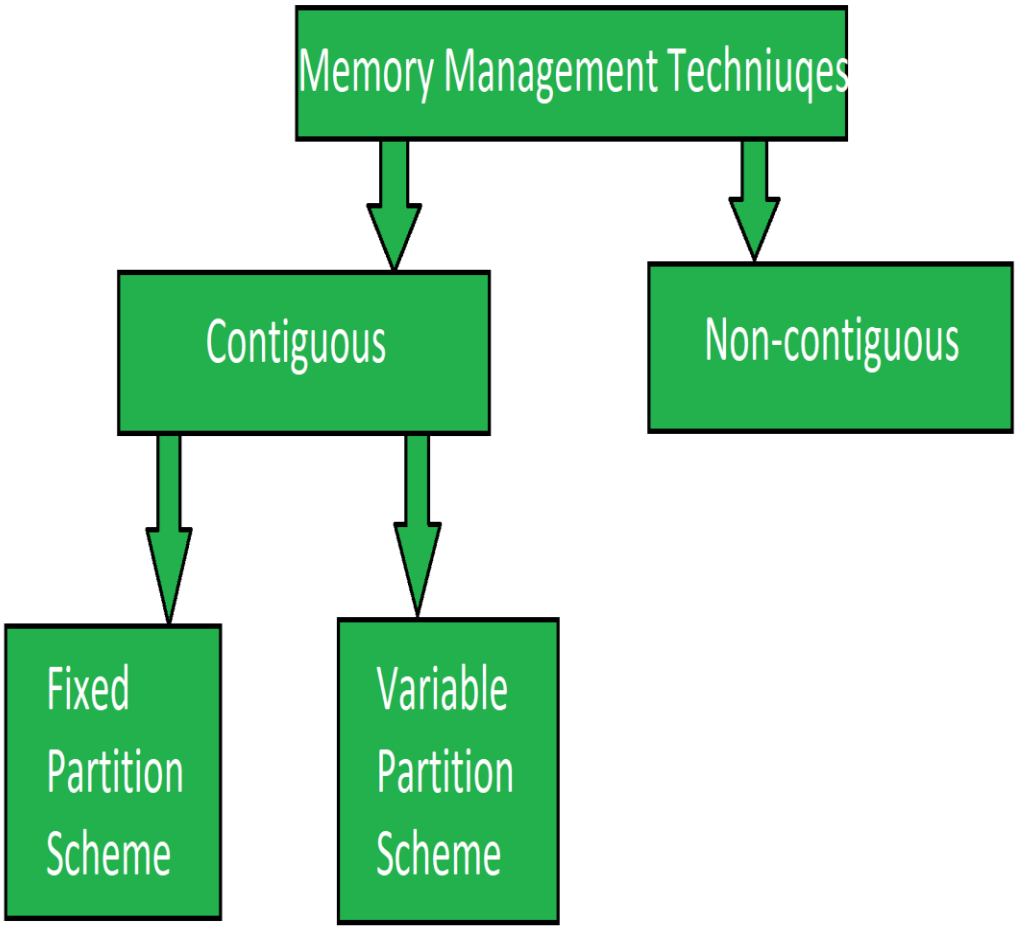
Implementation of Contiguous Memory Management Techniques

Memory Management Techniques are basic techniques that are used in managing the memory in operating system. Memory Management Techniques are basically classified into two categories:

```
(i) Contiguous
(ii) Non-contiguous
```

Contiguous Memory Management Techniques

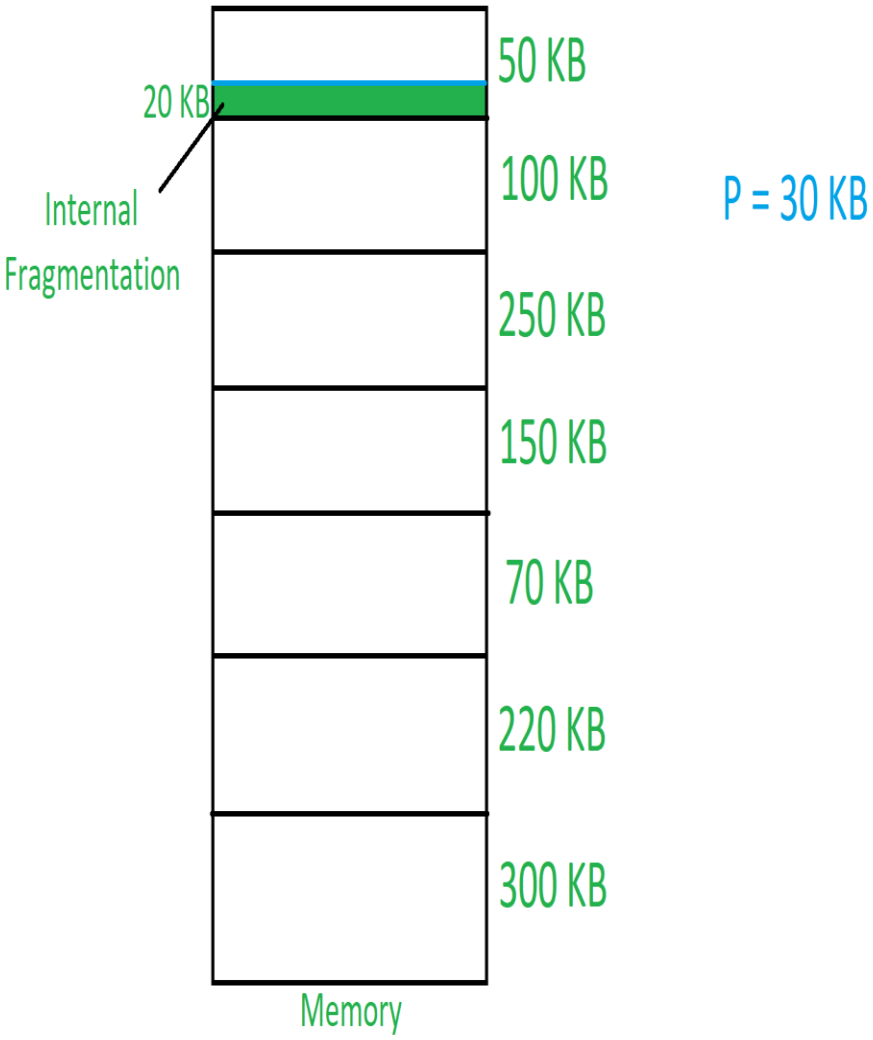
In this technique, memory is allotted in a continuous way to the processes.
It has two types:



Fixed Partition Scheme-

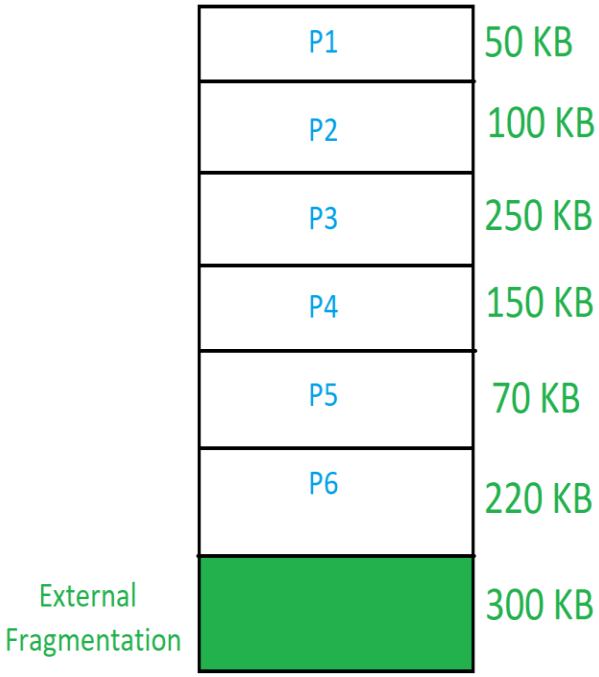
In the fixed partition scheme, memory is divided into fixed number of partitions. Fixed means number of partitions are fixed in the memory. In the fixed partition, in every partition only one process will be accommodated. Degree of multi-programming is restricted by number of partitions in the memory. Maximum size of the process is restricted by maximum size of the partition. Every partition is associated with the *limit registers*.

- **Limit Registers:** It has two limit:
- **Lower Limit:** Starting address of the partition.
- **Upper Limit:** Ending address of the partition.



Internal Fragmentation
is found in fixed partition scheme.To overcome the problem of internal fragmentation, instead of fixed partition scheme, variable partition scheme is used.

Variable Partition Scheme
In the variable partition scheme, initially memory will be single continuous free block. Whenever the request by the process arrives, accordingly partition will be made in the memory. If the smaller processes keep on coming then the larger partitions will be made into smaller partitions.



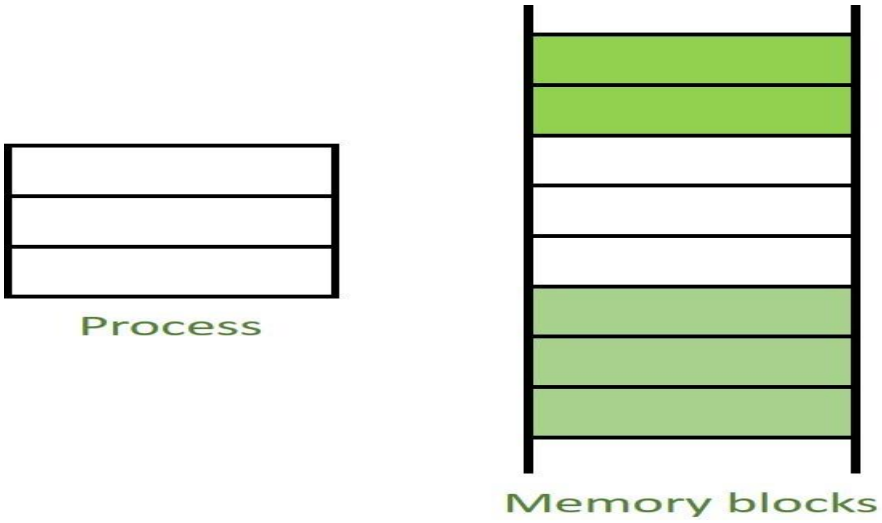
External Fragmentation is found in variable partition scheme.
To overcome the problem of external fragmentation, compaction technique is used or non-contiguous memory management techniques are used.

Compaction:
Moving all the processes toward the top or towards the bottom to make free available memory in a single continuous place is called as compaction. Compaction is undesirable to implement because it interrupts all the running processes in the memory.

Difference between Contiguous and Noncontiguous Memory Allocation

1. Contiguous Memory Allocation –

Contiguous memory allocation is basically a method in which a single contiguous section/part of memory is allocated to a process or file needing it. Because of this all the available memory space resides at the same place together, which means that the freely/unused available memory partitions are not distributed in a random fashion here and there across the whole memory space.



Contiguous Memory Allocation

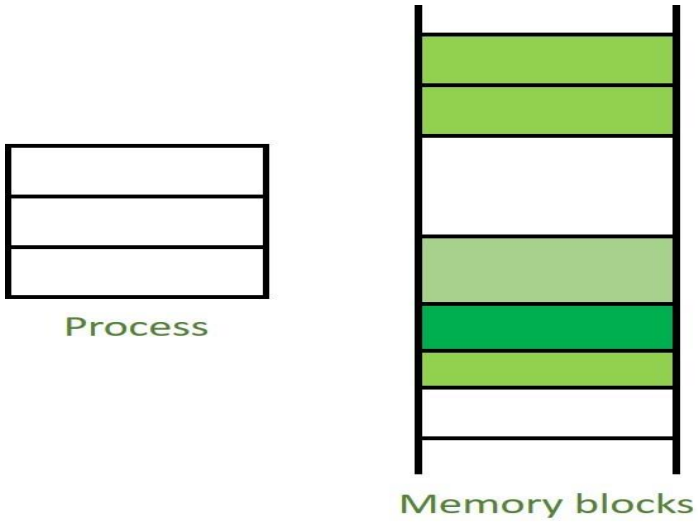
The main memory is a combination of two main portions- one for the operating system and other for the user program.

We can implement/achieve contiguous memory allocation by dividing the memory partitions into fixed size partitions.

2. Non-Contiguous Memory Allocation

Non-Contiguous memory allocation is basically a method on the contrary to contiguous allocation method, allocates the memory space present in different locations to the process as per it's requirements. As all the available memory space is in a distributed pattern so the freely available memory space is also scattered here and there.

This technique of memory allocation helps to reduce the wastage of memory, which eventually gives rise to Internal and external fragmentation.



Noncontiguous Memory Allocation

Difference between Contiguous and Non-contiguous Memory Allocation

S.NO.	CONTIGUOUS MEMORY ALLOCATION	NON-CONTIGUOUS MEMORY ALLOCATION
1.	Contiguous memory allocation allocates consecutive blocks of memory to a file/process.	Non-Contiguous memory allocation allocates separate blocks of memory to a file/process.
2.	Faster in	Slower in

S.NO.	CONTIGUOUS MEMORY ALLOCATION	NON-CONTIGUOUS MEMORY ALLOCATION
	Execution.	Execution.
3.	It is easier for the OS to control.	It is difficult for the OS to control.
4.	Overhead is minimum as not much address translations are there while executing a process.	More Overheads are there as there are more address translations.
5.	Internal fragmentation occurs in Contiguous memory	External fragmentation occurs in Non-Contiguous memory

MITRC

S.NO.	CONTIGUOUS MEMORY ALLOCATION	NON-CONTIGUOUS MEMORY ALLOCATION
	allocation method.	allocation method.
6.	It includes single partition allocation and multi-partition allocation.	It includes paging and segmentation.
7.	Wastage of memory is there.	No memory wastage is there.

MITRC

UNIT-2 (Operating System)
Virtual Memory
LECTURE NO:-2

Operating System - Virtual Memory

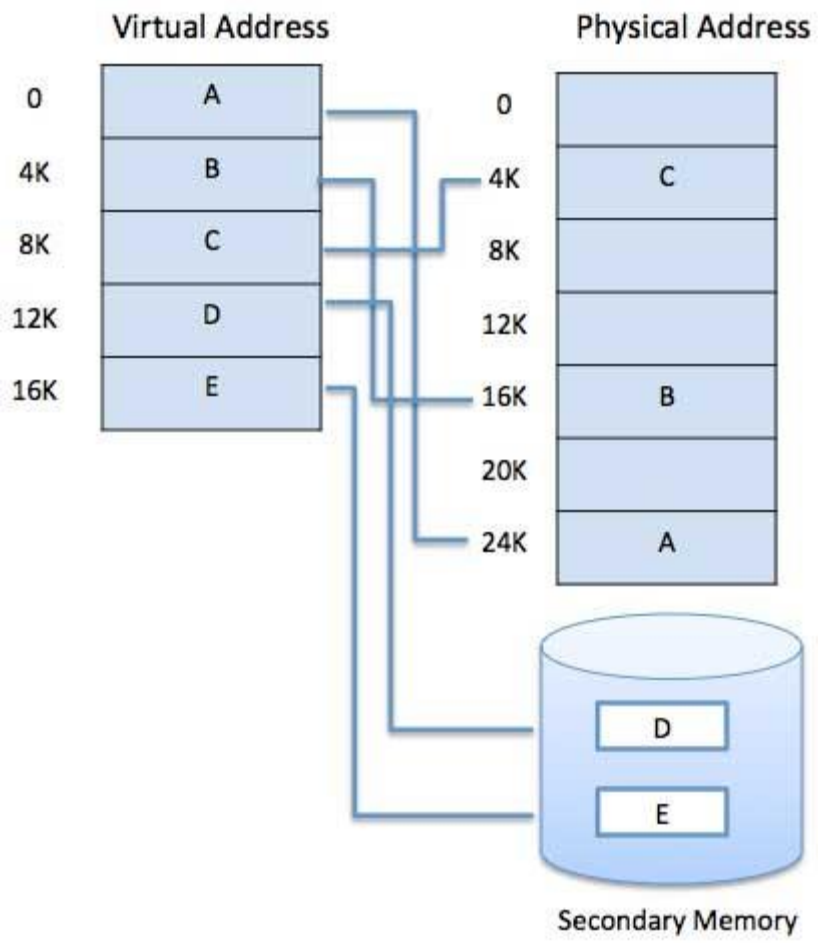
A computer can address more memory than the amount physically installed on the system. This extra memory is actually called **virtual memory** and it is a section of a hard disk that's set up to emulate the computer's RAM.

The main visible advantage of this scheme is that programs can be larger than physical memory. Virtual memory serves two purposes. First, it allows us to extend the use of physical memory by using disk. Second, it allows us to have memory protection, because each virtual address is translated to a physical address.

Following are the situations, when entire program is not required to be loaded fully in main memory.

- User written error handling routines are used only when an error occurred in the data or computation.
- Certain options and features of a program may be used rarely.
- Many tables are assigned a fixed amount of address space even though only a small amount of the table is actually used.
- The ability to execute a program that is only partially in memory would counter many benefits.
- Less number of I/O would be needed to load or swap each user program into memory.
- A program would no longer be constrained by the amount of physical memory that is available.
- Each user program could take less physical memory, more programs could be run the same time, with a corresponding increase in CPU utilization and throughput.

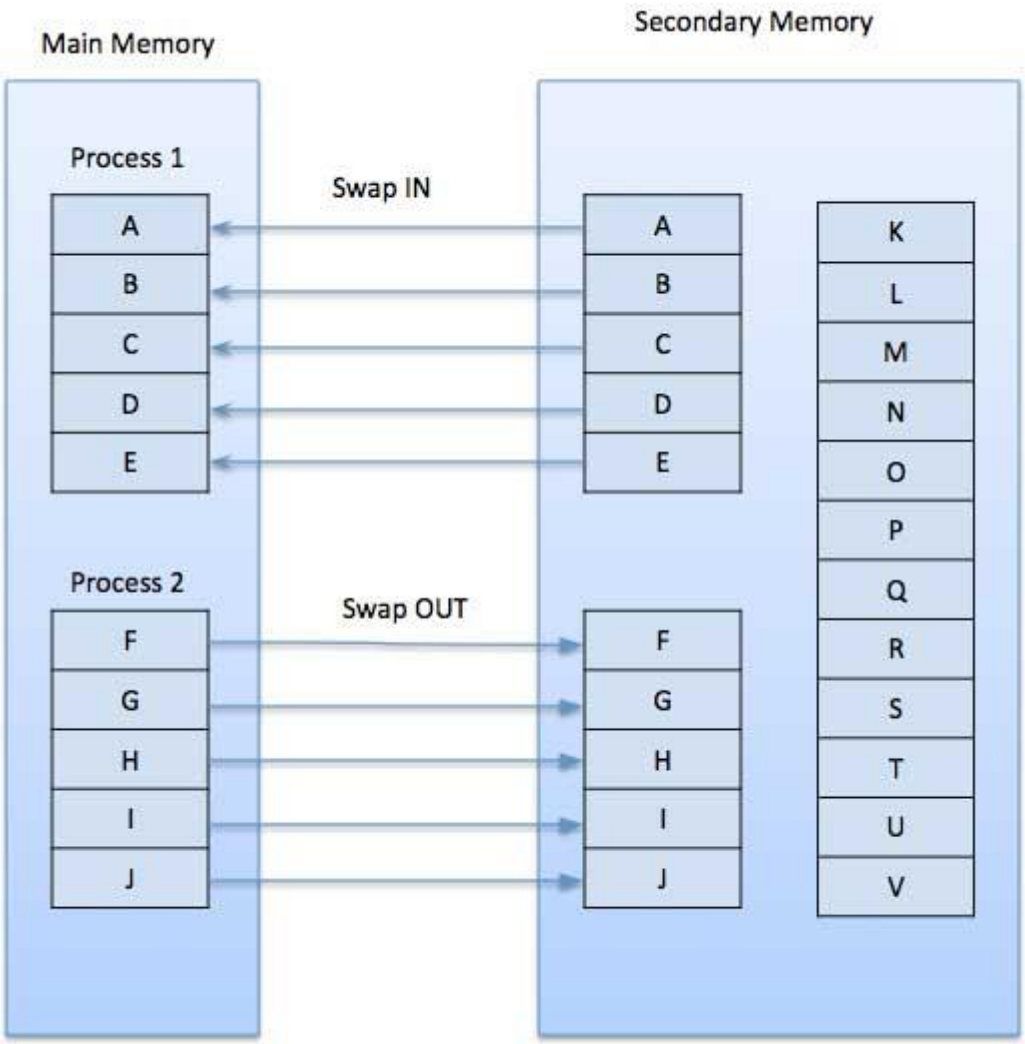
Modern microprocessors intended for general-purpose use, a memory management unit, or MMU, is built into the hardware. The MMU's job is to translate virtual addresses into physical addresses. A basic example is given below −



Virtual memory is commonly implemented by demand paging. It can also be implemented in a segmentation system. Demand segmentation can also be used to provide virtual memory.

Demand Paging

A demand paging system is quite similar to a paging system with swapping where processes reside in secondary memory and pages are loaded only on demand, not in advance. When a context switch occurs, the operating system does not copy any of the old program's pages out to the disk or any of the new program's pages into the main memory Instead, it just begins executing the new program after loading the first page and fetches that program's pages as they are referenced.



While executing a program, if the program references a page which is not available in the main memory because it was swapped out a little ago, the processor treats this invalid memory reference as a **page fault** and transfers control from the program to the operating system to demand the page back into the memory.

Advantages

Following are the advantages of Demand Paging −

- Large virtual memory.
- More efficient use of memory.
- There is no limit on degree of multiprogramming.

Disadvantages

Number of tables and the amount of processor overhead for handling page interrupts are greater than in the case of the simple paged management techniques.next →← prev

MITRC

UNIT-2(Operating System - Memory Management)
Demand Paging
LECTURE NO:-3

Demand Paging

According to the concept of Virtual Memory, in order to execute some process, only a part of the process needs to be present in the main memory which means that only a few pages will only be present in the main memory at any time.

However, deciding, which pages need to be kept in the main memory and which need to be kept in the secondary memory, is going to be difficult because we cannot say in advance that a process will require a particular page at particular time.

Therefore, to overcome this problem, there is a concept called Demand Paging is introduced. It suggests keeping all pages of the frames in the secondary memory until they are required. In other words, it says that do not load any page in the main memory until it is required.

Whenever any page is referred for the first time in the main memory, then that page will be found in the secondary memory.

After that, it may or may not be present in the main memory depending upon the page replacement algorithm which will be covered later in this tutorial.

What is a Page Fault?

If the referred page is not present in the main memory then there will be a miss and the concept is called Page miss or page fault.

The CPU has to access the missed page from the secondary memory. If the number of page fault is very high then the effective access time of the system will become very high.

What is Thrashing?

If the number of page faults is equal to the number of referred pages or the number of page faults are so high so that the CPU remains busy in just reading the pages from the secondary memory then the effective access time will be the time taken by the CPU to read one word from the secondary memory and it will be so high. The concept is called thrashing.

If the page fault rate is PF %, the time taken in getting a page from the secondary memory and again restarting is S (service time) and the memory access time is ma then the effective access time can be given as;

- EAT = PF X S + (1 - PF) X (ma)
-

Inverted Page Table

Inverted Page Table is the global page table which is maintained by the Operating System for all the processes. In inverted page table, the number of entries is equal to the number of frames in the main memory. It can be used to overcome the drawbacks of page table.

There is always a space reserved for the page regardless of the fact that whether it is present in the main memory or not. However, this is simply the wastage of the memory if the page is not present.

Pages	Frames
0	X
1	X
2	F1
3	F3
4	F6
5	X
6	F5

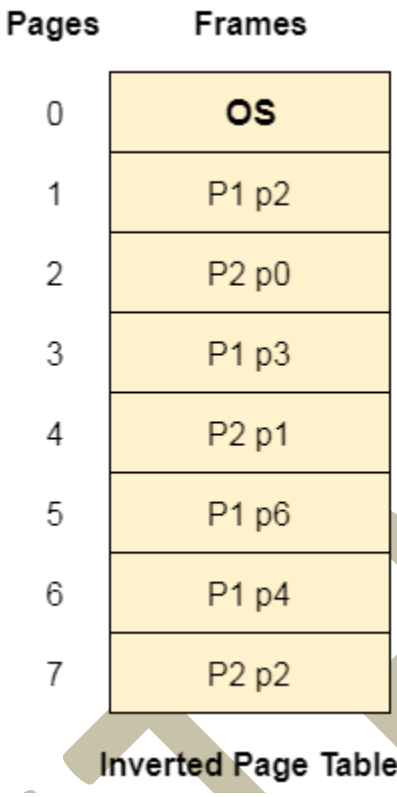
Page Table of P1

Pages	Frames
0	F2
1	F4
2	F7
3	X
4	X
5	X
6	F0

Page Table of P2

We can save this wastage by just inverting the page table. We can save the details only for the pages which are present in the main memory. Frames are the indices and the information saved inside the block will be Process ID and page number.

MITRC


Pages
Frames
0
OS
1
P1 p2
2
P2 p0
3
P1 p3
4
P2 p1
5
P1 p6
6
P1 p4
7
P2 p2
Inverted Page Table

MITRC

UNIT-1(INTRODUCTION OF OS, LAYERED ARCHITECTURE/ LOGICAL STRUCTURE OF OS)
Page Replacement Algorithm
LECTURE NO:-4

Page Replacement Algorithm

Page replacement algorithms are the techniques using which an Operating System decides which memory pages to swap out, write to disk when a page of memory needs to be allocated. Paging happens whenever a page fault occurs and a free page cannot be used for allocation purpose accounting to reason that pages are not available or the number of free pages is lower than required pages.

When the page that was selected for replacement and was paged out, is referenced again, it has to read in from disk, and this requires for I/O completion. This process determines the quality of the page replacement algorithm: the lesser the time waiting for page-ins, the better is the algorithm.

A page replacement algorithm looks at the limited information about accessing the pages provided by hardware, and tries to select which pages should be replaced to minimize the total number of page misses, while balancing it with the costs of primary storage and processor time of the algorithm itself. There are many different page replacement algorithms. We evaluate an algorithm by running it on a particular string of memory reference and computing the number of page faults,

Reference String

The string of memory references is called reference string. Reference strings are generated artificially or by tracing a given system and recording the address of each memory reference. The latter choice produces a large number of data, where we note two things.

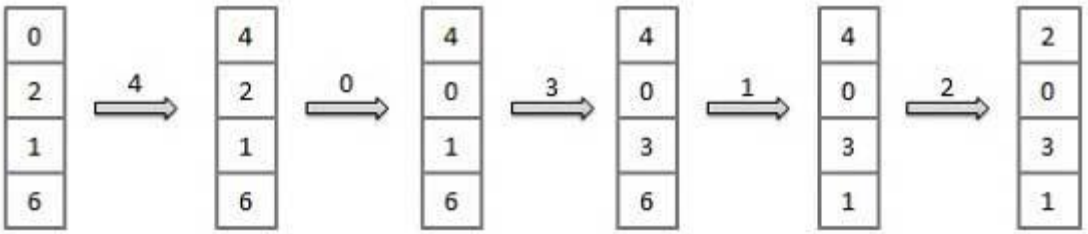
- For a given page size, we need to consider only the page number, not the entire address.
- If we have a reference to a page **p**, then any immediately following references to page **p** will never cause a page fault. Page p will be in memory after the first reference; the immediately following references will not fault.
- For example, consider the following sequence of addresses − 123,215,600,1234,76,96
- If page size is 100, then the reference string is 1,2,6,12,0,0
-

First In First Out (FIFO) algorithm

- Oldest page in main memory is the one which will be selected for replacement.
- Easy to implement, keep a list, replace pages from the tail and add new pages at the head.

Reference String : 0, 2, 1, 6, 4, 0, 1, 0, 3, 1, 2, 1

Misses : x x x x x x x x x



Fault Rate = 9 / 12 = 0.75

Optimal Page algorithm

- An optimal page-replacement algorithm has the lowest page-fault rate of all algorithms. An optimal page-replacement algorithm exists, and has been called OPT or MIN.
- Replace the page that will not be used for the longest period of time. Use the time when a page is to be used.

Paging | Memory Management | Operating System

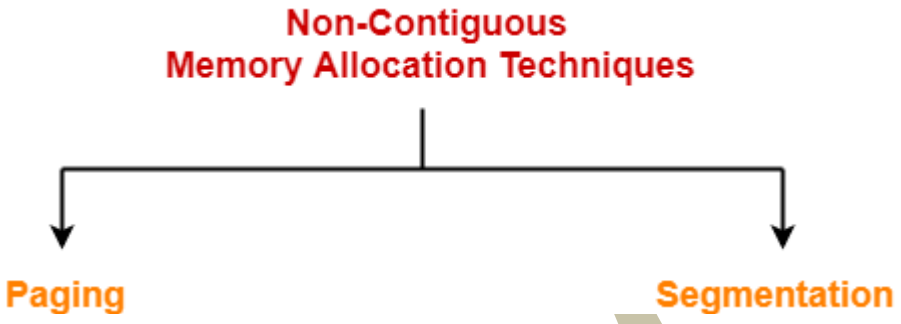
Non-Contiguous Memory Allocation-

- Non-contiguous memory allocation is a memory allocation technique.
- It allows to store parts of a single process in a non-contiguous fashion.
- Thus, different parts of the same process can be stored at different places in the main memory.

Techniques-

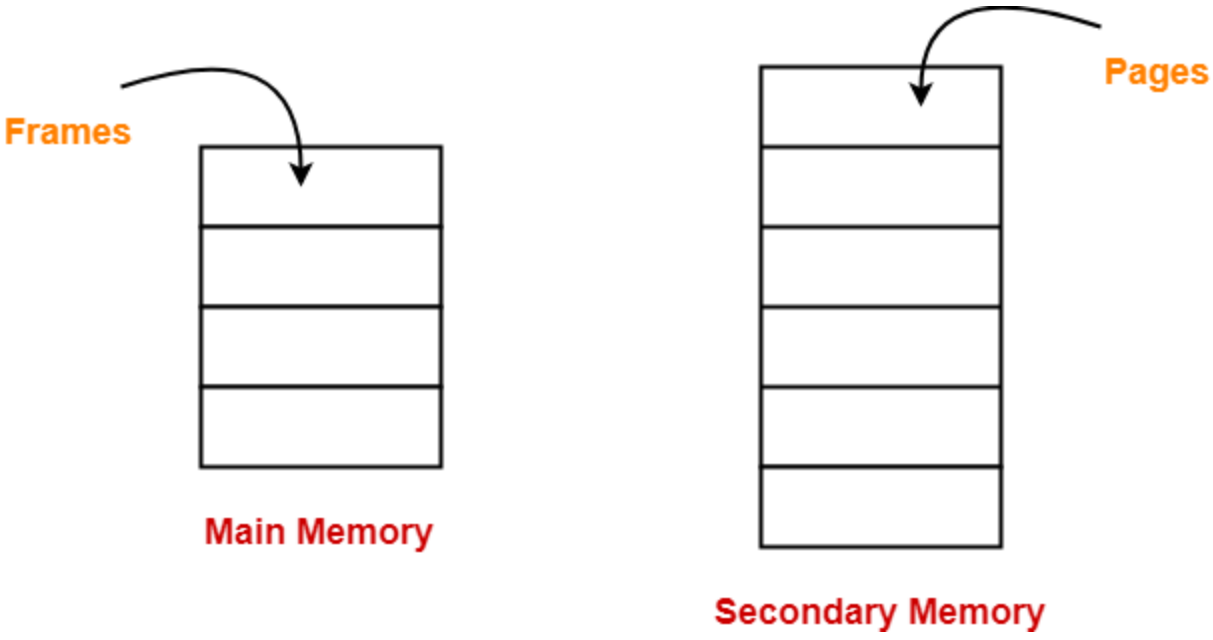
There are two popular techniques used for non-contiguous memory allocation-

1. Paging
2. Segmentation



Paging-

- Paging is a fixed size partitioning scheme.
- In paging, secondary memory and main memory are divided into equal fixed size partitions.
- The partitions of secondary memory are called as **pages**.
- The partitions of main memory are called as **frames**.

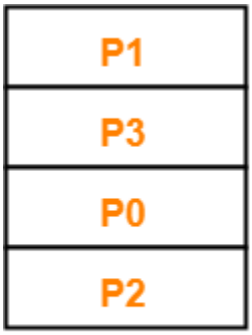


- Each process is divided into parts where size of each part is same as page size.
- The size of the last part may be less than the page size.

- The pages of process are stored in the frames of main memory depending upon their availability.

Example-

- Consider a process is divided into 4 pages P_0, P_1, P_2 and P_3.
- Depending upon the availability, these pages may be stored in the main memory frames in a non-contiguous fashion as shown-



Main Memory

Also Read- Contiguous Memory Allocation

Translating Logical Address into Physical Address-

- CPU always generates a logical address.
- A physical address is needed to access the main memory.

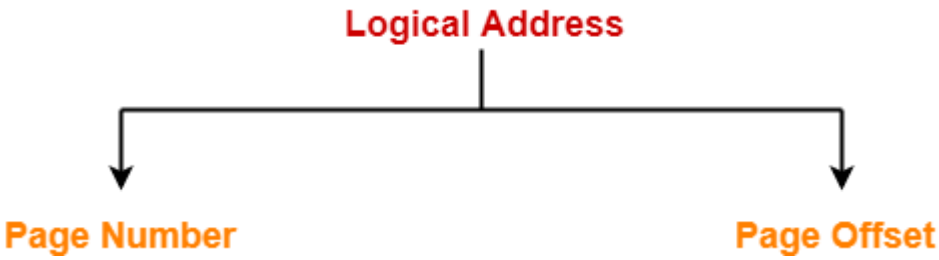
Following steps are followed to translate logical address into physical address-

Step-01:

CPU generates a logical address consisting of two parts-

1. Page Number

2. Page Offset



- Page Number specifies the specific page of the process from which CPU wants to read the data.
- Page Offset specifies the specific word on the page that CPU wants to read.

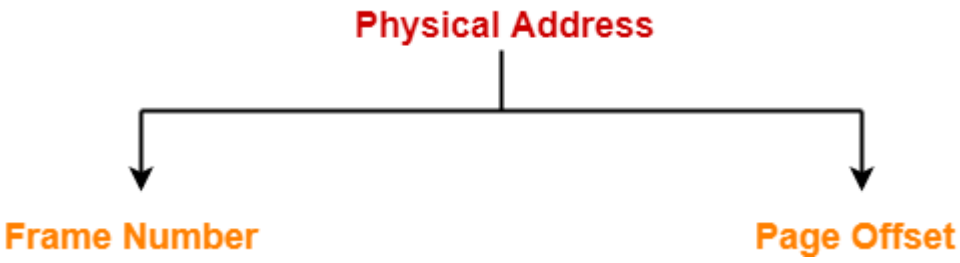
Step-02:

For the page number generated by the CPU,

- **Page Table** provides the corresponding frame number (base address of the frame) where that page is stored in the main memory.

Step-03:

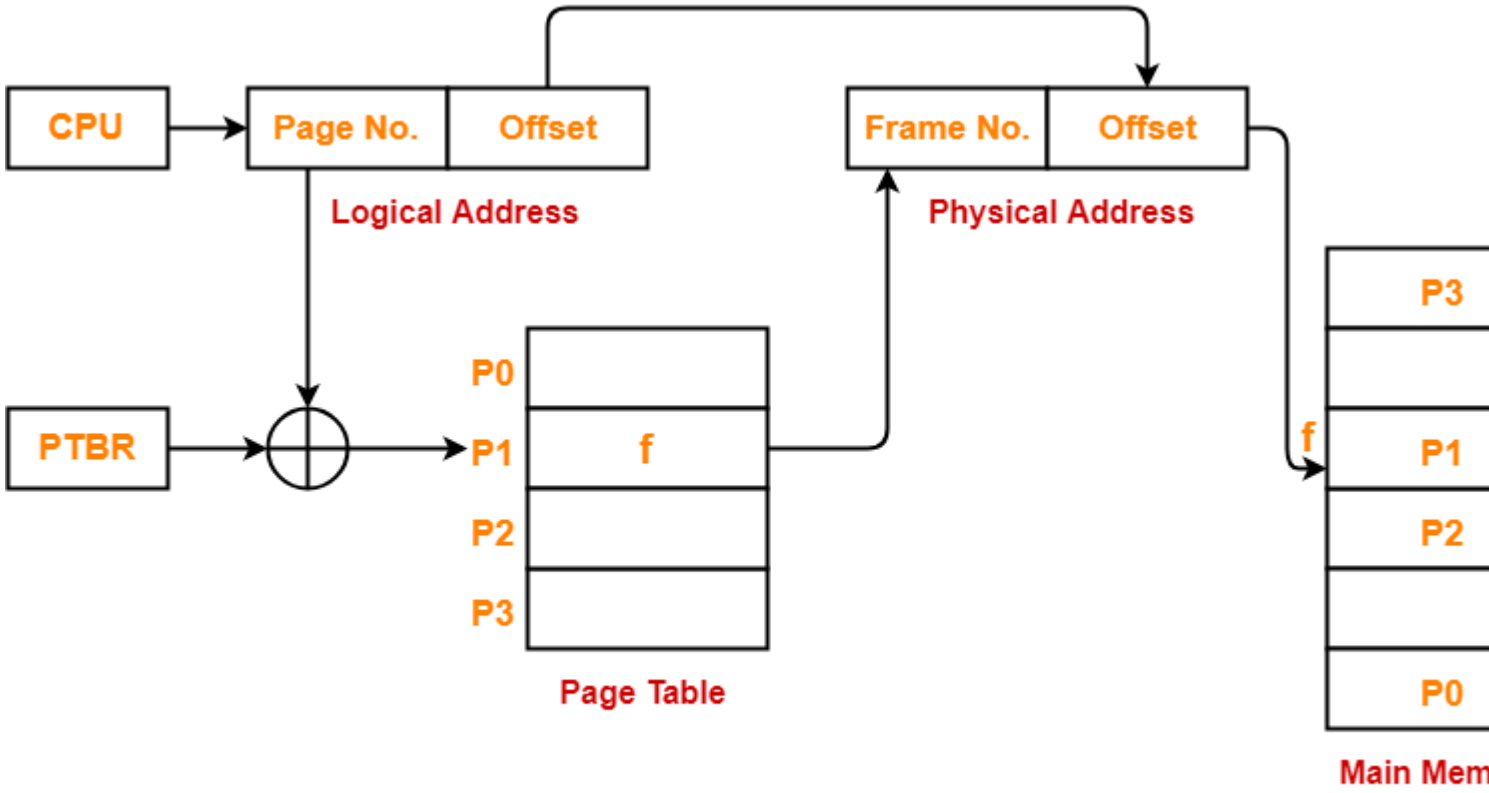
- The frame number combined with the page offset forms the required physical address.



- Frame number specifies the specific frame where the required page is stored.
- Page Offset specifies the specific word that has to be read from that page.

Diagram-

The following diagram illustrates the above steps of translating logical address into physical address-



Translating Logical Address into Physical Address

Advantages-

The advantages of paging are-

- It allows to store parts of a single process in a non-contiguous fashion.
- It solves the problem of external fragmentation.

Disadvantages-

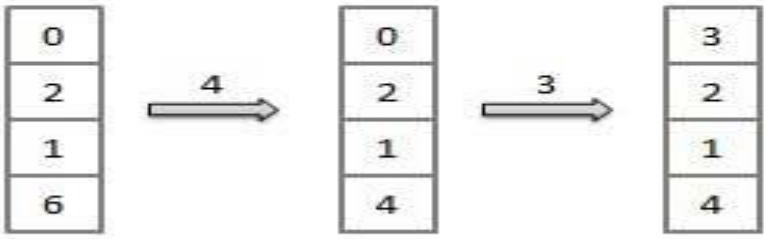
The disadvantages of paging are-

- It suffers from internal fragmentation.

- There is an overhead of maintaining a page table for each process.
- The time taken to fetch the instruction increases since now two memory accesses are required.

Reference String : 0, 2, 1, 6, 4, 0, 1, 0, 3, 1, 2, 1

Misses : x x x x x x



Fault Rate = 6 / 12 = 0.50

UNIT-1(INTRODUCTION OF OS, LAYERED ARCHITECTURE/ LOGICAL STRUCTURE OF OS)

- **Process Creation**

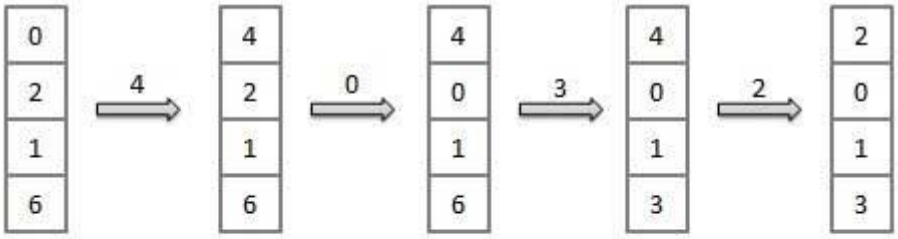
LECTURE NO:-5

Least Recently Used (LRU) algorithm

- Page which has not been used for the longest time in main memory is the one which will be selected for replacement.
- Easy to implement, keep a list, replace pages by looking back into time.

Reference String : 0, 2, 1, 6, 4, 0, 1, 0, 3, 1, 2, 1

Misses : x x x x x x x x



Fault Rate = 8 / 12 = 0.67

Page Buffering algorithm

- To get a process start quickly, keep a pool of free frames.
- On page fault, select a page to be replaced.
- Write the new page in the frame of free pool, mark the page table and restart the process.
- Now write the dirty page out of disk and place the frame holding replaced page in free pool.

Least frequently Used (LFU) algorithm

- The page with the smallest count is the one which will be selected for replacement.
- This algorithm suffers from the situation in which a page is used heavily during the initial phase of a process, but then is never used again.

Most frequently Used (MFU) algorithm

- This algorithm is based on the argument that the page with the smallest count was probably just brought in and has yet to be used.

Page Table | Paging in Operating System

We have discussed-

- Paging is a non-contiguous memory allocation technique.
- The logical address generated by the CPU is translated into the physical address using the page table.

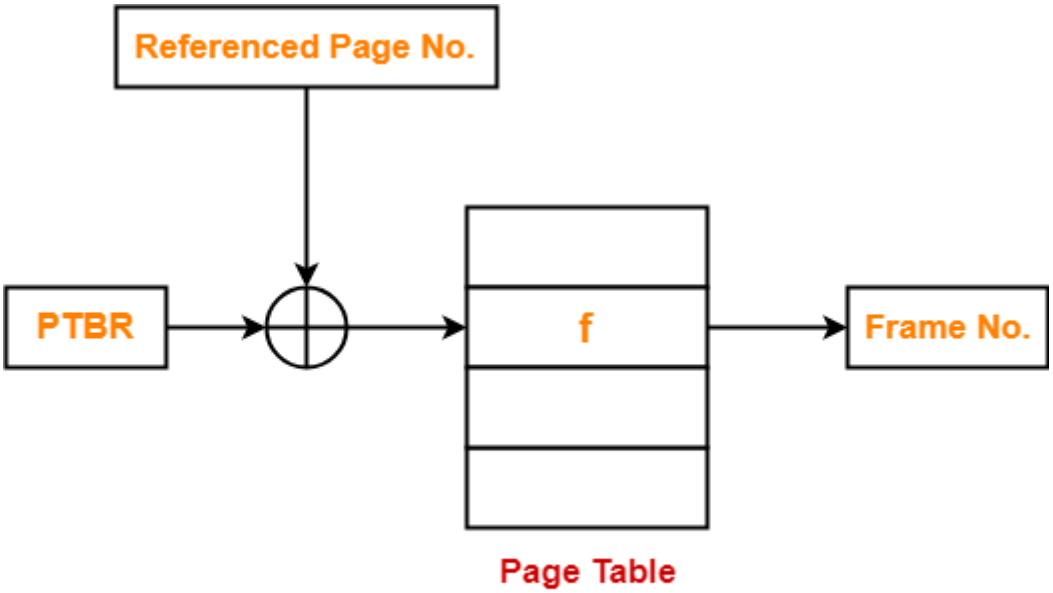
Page Table-

- Page table is a data structure.
- It maps the page number referenced by the CPU to the frame number where that page is stored.

Characteristics-

- Page table is stored in the main memory.
- Number of entries in a page table = Number of pages in which the process is divided.
- Page Table Base Register (PTBR) contains the base address of page table.
- Each process has its own independent page table.

Working-



Obtaining Frame Number Using Page Table

- Page Table Base Register (PTBR) provides the base address of the page table.
- The base address of the page table is added with the page number referenced by the CPU.
- It gives the entry of the page table containing the frame number where the referenced page is stored.

Page Table Entry-

- A page table entry contains several information about the page.
- The information contained in the page table entry varies from operating system to operating system.
- The most important information in a page table entry is frame number.

In general, each entry of a page table contains the following information-



Page Table Entry Format

1. Frame Number-

- Frame number specifies the frame where the page is stored in the main memory.
- The number of bits in frame number depends on the number of frames in the main memory.

2. Present / Absent Bit-

- This bit is also sometimes called as **valid / invalid bit**.
- This bit specifies whether that page is present in the main memory or not.
- If the page is not present in the main memory, then this bit is set to 0 otherwise set to 1.

NOTE

- If the required page is not present in the main memory, then it is called as **Page Fault**.
- A page fault requires page initialization.
- The required page has to be initialized (fetched) from the secondary memory and brought into the r memory.

3. Protection Bit-

- This bit is also sometimes called as "**Read / Write bit**".
- This bit is concerned with the page protection.
- It specifies the permission to perform read and write operation on the page.
- If only read operation is allowed to be performed and no writing is allowed, then this bit is set to 0.

- If both read and write operation are allowed to be performed, then this bit is set to 1.

4. Reference Bit-

- Reference bit specifies whether that page has been referenced in the last clock cycle or not.
- If the page has been referenced recently, then this bit is set to 1 otherwise set to 0.

NOTE

- Reference bit is useful for page replacement policy.
- A page that has not been referenced recently is considered a good candidate for page replacement LRU page replacement policy.

5. Caching Enabled / Disabled-

- This bit enables or disables the caching of page.
- Whenever freshness in the data is required, then caching is disabled using this bit.
- If caching of the page is disabled, then this bit is set to 1 otherwise set to 0.

6. Dirty Bit-

- This bit is also sometimes called as "**Modified bit**".
- This bit specifies whether that page has been modified or not.
- If the page has been modified, then this bit is set to 1 otherwise set to 0.

NOTE

In case the page is modified,

- Before replacing the modified page with some other page, it has to be written back in the secondary memory to avoid losing the data.
- Dirty bit helps to avoid unnecessary writes.
- This is because if the page is not modified, then it can be directly replaced by another page without need of writing it back to the disk.

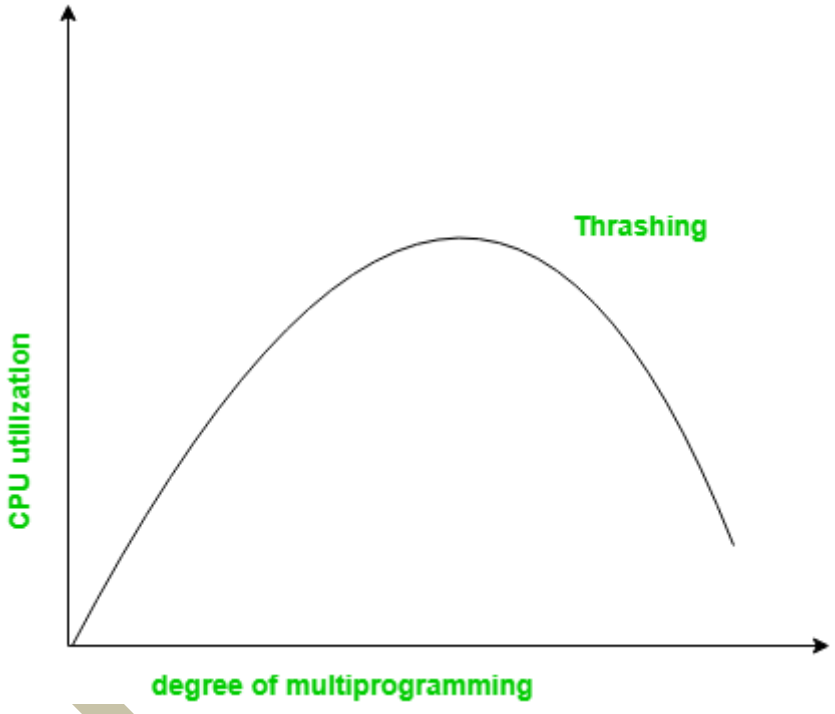
MITRC

UNIT-1(INTRODUCTION OF OS, LAYERED ARCHITECTURE/ LOGICAL STRUCTURE OF OS)
Process Creation
LECTURE NO:-6

Thrashing

Prerequisite – Virtual Memory

Thrashing is a condition or a situation when the system is spending a major portion of its time in servicing the page faults, but the actual processing done is very negligible.



The basic concept involved is that if a process is allocated too few frames, then there will be too many and too frequent page faults. As a result, no useful work would be done by the CPU and the CPU utilisation would fall drastically. The long-term scheduler would then try to improve the CPU utilisation by loading some more processes into the memory thereby increasing the degree of multiprogramming. This would result in a further decrease in the CPU utilization triggering a chained reaction of higher page faults followed by an increase in the degree of multiprogramming, called Thrashing.

Locality Model –

A locality is a set of pages that are actively used together. The locality model states that as a process executes, it moves from one locality to another. A program is generally composed of several different localities which may overlap.

For example when a function is called, it defines a new locality where memory references are made to the instructions of the function call, it's local and global variables, etc. Similarly, when the function is exited, the process leaves this locality.

Techniques to handle

1. **Working Set Model –**

   This model is based on the above-stated concept of the Locality Model. The basic principle states that if we allocate enough frames to a process to accommodate its current locality, it will only fault whenever it moves to some new locality. But if the allocated frames are lesser than the size of the current locality, the process is bound to thrash.

   According to this model, based on a parameter A, the working set is defined as the set of pages in the most recent 'A' page references. Hence, all the actively used pages would always end up being a part of the working set.

   The accuracy of the working set is dependant on the value of parameter A. If A is too large, then working sets may overlap. On the other hand, for smaller values of A, the locality might not be covered entirely.

   If D is the total demand for frames and is the working set size for a process

   Now, if 'm' is the number of frames available in the memory, there are 2 possibilities:

   - (i) D>m i.e. total demand exceeds the number of frames, then thrashing will occur as some processes would not get enough frames.
   - (ii) D<=m, then there would be no thrashing.

   If there are enough extra frames, then some more processes can be loaded in the memory. On the other hand, if the summation of working set sizes exceeds the availability of frames, then some of the processes have to be suspended(swapped out of memory).

The basic concept involved is that if a process is allocated too few frames, then there will be too many and too frequent page faults. As a result, no useful work would be done by the CPU and the CPU utilisation would fall drastically. The long-term scheduler would then try to improve the CPU utilisation by loading some more processes into the memory thereby

increasing the degree of multiprogramming. This would result in a further decrease in the CPU utilization triggering a chained reaction of higher page faults followed by an increase in the degree of multiprogramming, called Thrashing.

Locality Model –

A locality is a set of pages that are actively used together. The locality model states that as a process executes, it moves from one locality to another. A program is generally composed of several different localities which may overlap.
For example when a function is called, it defines a new locality where memory references are made to the instructions of the function call, it's local and global variables, etc. Similarly, when the function is exited, the process leaves this locality.

Techniques to handle:

1. **Working Set Model –**

   This model is based on the above-stated concept of the Locality Model. The basic principle states that if we allocate enough frames to a process to accommodate its current locality, it will only fault whenever it moves to some new locality. But if the allocated frames are lesser than the size of the current locality, the process is bound to thrash.

   According to this model, based on a parameter A, the working set is defined as the set of pages in the most recent 'A' page references. Hence, all the actively used pages would always end up being a part of the working set.

   The accuracy of the working set is dependant on the value of parameter A. If A is too large, then working sets may overlap. On the other hand, for smaller values of A, the locality might not be covered entirely.

   If D is the total demand for frames and is the working set size for a process i,

   Now, if 'm' is the number of frames available in the memory, there are 2 possibilities:

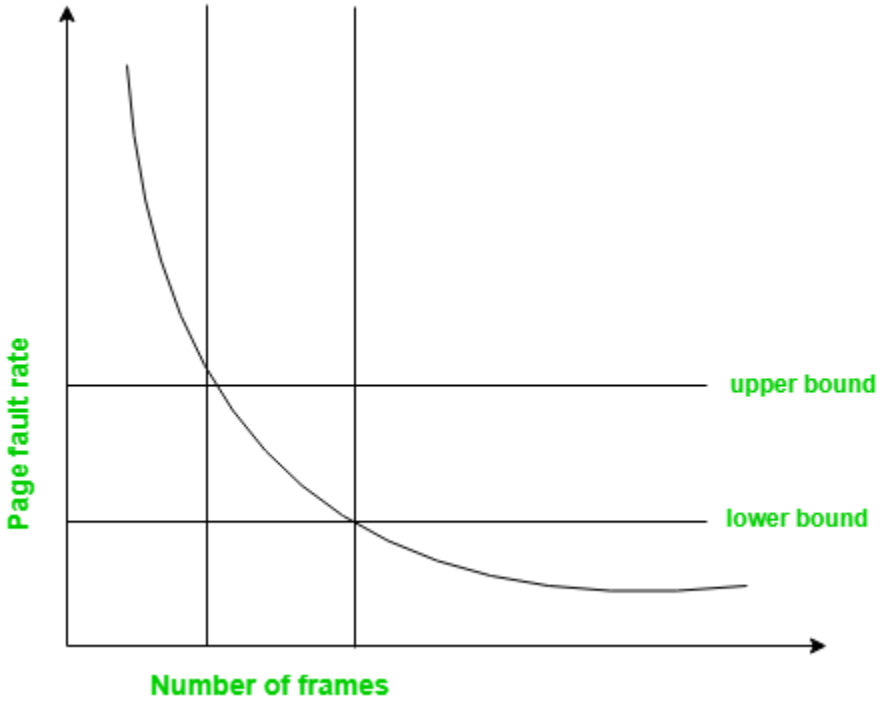
   - (i) D>m i.e. total demand exceeds the number of frames, then thrashing will occur as some processes would not get enough frames.
   - (ii) D<=m, then there would be no thrashing.

   If there are enough extra frames, then some more processes can be loaded in the memory. On the other hand, if the summation of working set sizes exceeds the availability of frames, then some of the processes have to be suspended(swapped out of memory).

This technique prevents thrashing along with ensuring the highest degree of multiprogramming possible. Thus, it optimizes CPU utilisation.

2. **Page Fault Frequency –**

A more direct approach to handle thrashing is the one that uses Page-Fault Frequency concept.



The problem associated with Thrashing is the high page fault rate and thus, the concept here is to control the page fault rate.

If the page fault rate is too high, it indicates that the process has too few frames allocated to it. On the contrary, a low page fault rate indicates that the process has too many frames.

Upper and lower limits can be established on the desired page fault rate as shown in the diagram.
If the page fault rate falls below the lower limit, frames can be removed from the process. Similarly, if the page fault rate exceeds the upper limit, more number of frames can be allocated to the process.
In other words, the graphical state of the system should be kept limited to the rectangular region formed in the given diagram.

Here too, if the page fault rate is high with no free frames, then some of the processes can be suspended and frames allocated to them can be reallocated to other processes.

The suspended processes can then be restarted later.

MITRC

UNIT-2(INTRODUCTION OF OS, LAYERED ARCHITECTURE/ LOGICAL STRUCTURE OF OS)
Segmentation
LECTURE NO:-7

Why Segmentation is required?

In the Operating System, an important drawback of memory management is the separation of the user's view of memory and the actual physical memory. Paging is the scheme which provides the separation of these two memories.

The user's view is mapped onto the physical storage. This mapping permits differentiation between logical memory and physical memory.

Operating System may divide the same function into different pages and those pages may or may not be loaded at the same time into the memory and it doesn't care about the User's view of the process. This technique decreases the efficiency of the system.

Segmentation is better than this because it divides the process into the segments.

What is Segmentation?

Segmentation is a memory management technique which supports user's view of memory. This technique of division of a computer's primary memory into sections called segments.

Types of Segmentation

1. **Virtual memory segmentation**
   Each processor job is divided into several segments, It is not essential all of which are resident at any one point in time.
2. **Simple segmentation**
   Each process is divided into many segments, and all segments are loaded into the memory at run time, but not necessarily contiguously.

Basic method for Segmentation

In a computer system using segmentation, a logical address space can be viewed as multiple segments. The size of the segment may grow or shrink that is it is of variable length.

During execution, each segment has a name and a length. The address specifies both the segment name and the displacement within the segment. The user, therefore, specifies each address by two quantities; segment name and an offset.

Normally it is implemented as segments are numbered and are referred to by a segment number, in place of a segment name. Thus a logical address consists of two tuples:

< segment – number, offset >

Segment number(s) – It is the total number of bits required to represent the segment.

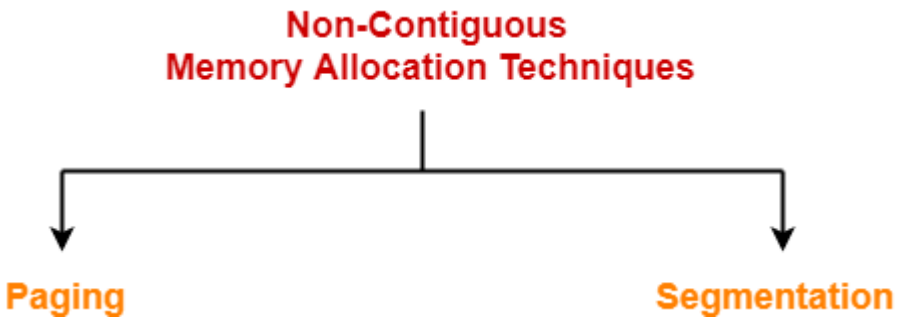
Segment Offset(d) – It specifies the number of bits required to represent the size of the segment.

We have discussed-

- Non-contiguous memory allocation is a memory allocation technique.
- It allows to store parts of a single process in a non-contiguous fashion.

Techniques-

There are two popular techniques used for non-contiguous memory allocation-



In this article, we will discuss about Segmentation.

Segmentation-

- Like Paging, Segmentation is another non-contiguous memory allocation technique.
- In segmentation, process is not divided blindly into fixed size pages.

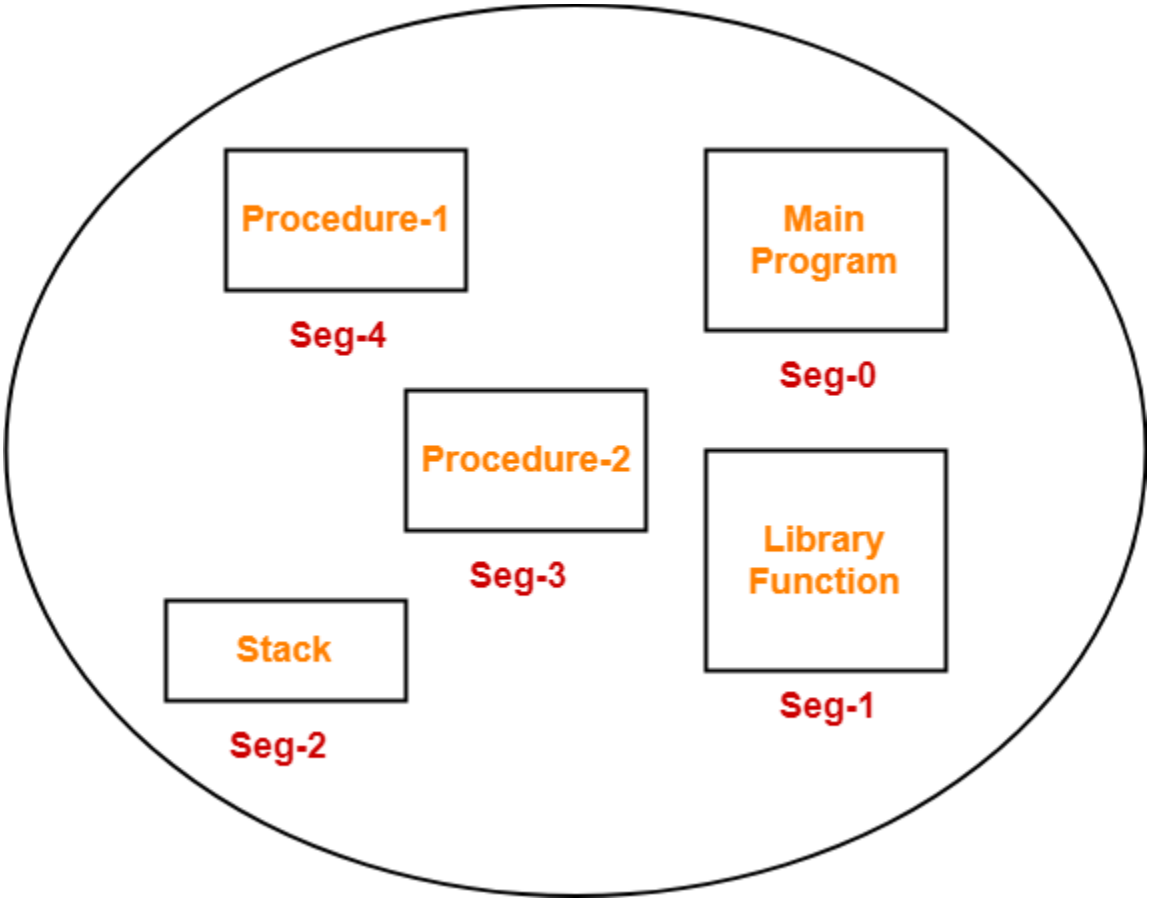
- Rather, the process is divided into modules for better visualization.

Characteristics-

- Segmentation is a variable size partitioning scheme.
- In segmentation, secondary memory and main memory are divided into partitions of unequal size.
- The size of partitions depend on the length of modules.
- The partitions of secondary memory are called as **segments**.

Example-

Consider a program is divided into 5 segments as-



Segment Table-

- Segment table is a table that stores the information about each segment of the process.
- It has two columns.
- First column stores the size or length of the segment.
- Second column stores the base address or starting address of the segment in the main memory.
- Segment table is stored as a separate segment in the main memory.
- Segment table base register (STBR) stores the base address of the segment table.

For the above illustration, consider the segment table is-

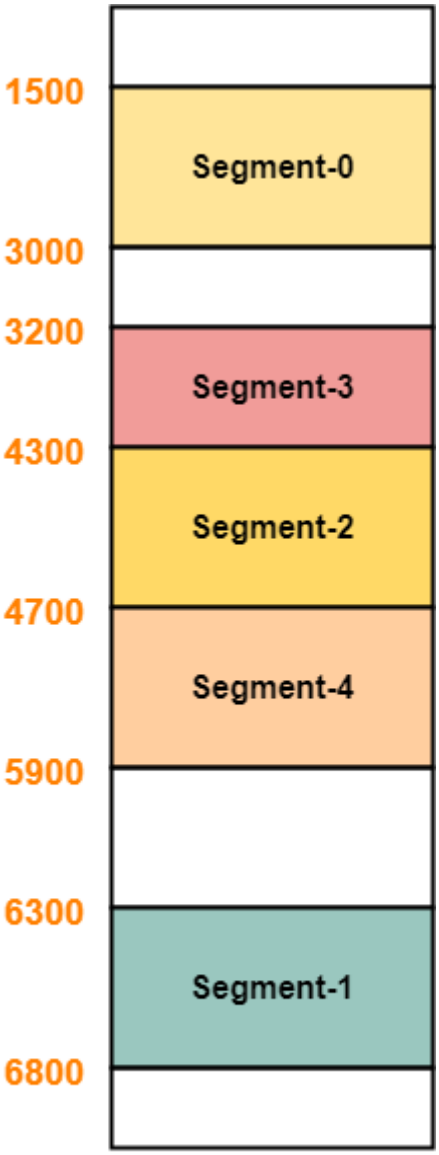
	Limit	Base
Seg-0	1500	1500
Seg-1	500	6300
Seg-2	400	4300
Seg-3	1100	3200
Seg-4	1200	4700

Segment Table

Here,

- Limit indicates the length or size of the segment.
- Base indicates the base address or starting address of the segment in the main memory.

In accordance to the above segment table, the segments are stored in the main memory as-


1500
Segment-0
3000
3200
Segment-3
4300
Segment-2
4700
Segment-4
5900
6300
Segment-1
6800
Main Memory

UNIT-2(INTRODUCTION OF OS, LAYERED ARCHITECTURE/ LOGICAL STRUCTURE OF OS)
Translating Logical Address into Physical Address-
LECTURE NO:- 8

Translating Logical Address into Physical Address-

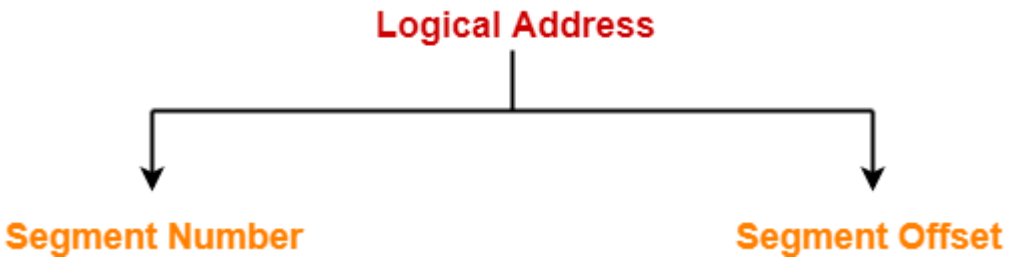
- CPU always generates a logical address.
- A physical address is needed to access the main memory.

Following steps are followed to translate logical address into physical address-

Step-01:

CPU generates a logical address consisting of two parts-

1. Segment Number
2. Segment Offset



- Segment Number specifies the specific segment of the process from which CPU wants to read the data.
- Segment Offset specifies the specific word in the segment that CPU wants to read.

Step-02:

- For the generated segment number, corresponding entry is located in the segment table.
- Then, segment offset is compared with the limit (size) of the segment.

Now, two cases are possible-

Case-01: Segment Offset >= Limit

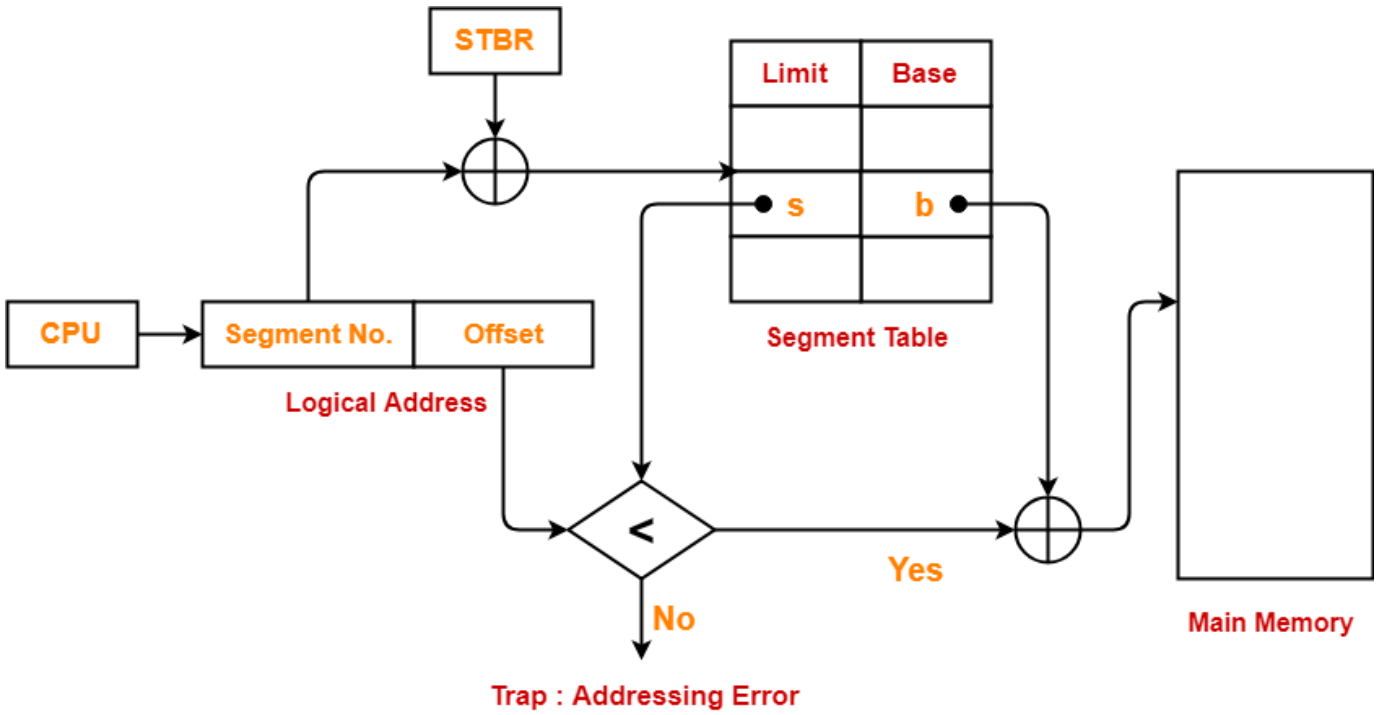
- If segment offset is found to be greater than or equal to the limit, a trap is generated.

Case-02: Segment Offset < Limit

- If segment offset is found to be smaller than the limit, then request is treated as a valid request.
- The segment offset must always lie in the range [0, limit-1],
- Then, segment offset is added with the base address of the segment.
- The result obtained after addition is the address of the memory location storing the required word.

Diagram-

The following diagram illustrates the above steps of translating logical address into physical address-



Translating Logical Address into Physical Address

Advantages-

The advantages of segmentation are-

- It allows to divide the program into modules which provides better visualization.
- Segment table consumes less space as compared to **Page Table** in paging.
- It solves the problem of internal fragmentation.

Disadvantages-

The disadvantages of segmentation are-

- There is an overhead of maintaining a segment table for each process.
- The time taken to fetch the instruction increases since now two memory accesses are required.
- Segments of unequal size are not suited for swapping.
- It suffers from external fragmentation as the free space gets broken down into smaller pieces with the processes being loaded and removed from the main memory.

Difference Between Paging and Segmentation in OS

The memory management in the operating system is an essential functionality, which allows the allocation of memory to the processes for execution and deallocates the memory when the

process is no longer needed. In this article, we will discuss two memory management schemes paging and segmentation.

he basic difference between paging and segmentation is that, “page” is a fixed-sized block whereas, a “segment” is a variable-sized block. We will discuss some more differences between Paging and Segmentation with the help of comparison chart shown below.

Content: Paging Vs Segmentation

1. Comparison Chart
2. Definition
3. Key Differences
4. Conclusion

Comparison Chart

BASIS FOR COMPARISON	PAGING	SEGMENTATION
Basic	A page is of fixed block size.	A segment is of variable size.
Fragmentation	Paging may lead to internal fragmentation.	Segmentation may lead to external fragmentation.
Address	The user specified address is divided by CPU into a page	The user specifies each address by two quantities a segment number and the

BASIS FOR COMPARISON	PAGING	SEGMENTATION
	number and offset.	offset (Segment limit).
Size	The hardware decides the page size.	The segment size is specified by the user.
Table	Paging involves a page table that contains base address of each page.	Segmentation involves the segment table that contains segment number and offset (segment length).

Key Differences Between Paging and Segmentation

1. The basic difference between paging and segmentation is that a page is always of **fixed block size** whereas, a segment is of **variable size**.
2. Paging may lead to **internal fragmentation** as the page is of fixed block size, but it may happen that the process does not acquire the entire block size which will generate the internal fragment in memory. The segmentation may lead to **external fragmentation** as the memory is filled with the variable sized blocks.
3. In paging the user only provides a **single integer** as the address which is divided by the hardware into a **page number and Offset**. On the other hands, in segmentation the user specifies the address in two quantities i.e. **segment number and offset**.
4. The size of the page is decided or specified by the **hardware**. On the other hands, the size of the segment is specified by the **user**.
5. In paging, the **page table** maps the **logical address to the physical address**, and it contains base address of each page stored in the frames of physical memory space.

However, in segmentation, the **segment table** maps the **logical address to the physical address**, and it contains segment number and offset (segment limit).

Conclusion

Paging and segmentation both are the **memory management schemes**. Paging allows the memory to be divided into **fixed sized block** whereas the segmentation, divides the memory space into **segments of the variable block size**. Where the paging leads to **internal fragmentation** the segmentation leads to **external fragmentation.**

MITRC